

PROYECTO INTERMODULAR DESARROLLO DE APLICACIONES MULTIPLATAFORMA



1. Introducción	3
- Contexto	3
- Objetivo	3
2. Análisis	4
- Problema	4
- Soluciones	5
- Utilidad de la aplicación	6
3. Tecnologías utilizadas	6
4. Diseño del sistema	8
- Diagrama Entidad/Relación	8
- Diagrama de clases	9
- Diagrama de Gantt	10
- Diagrama de actividades	11
5. Pruebas	11
6. Implementación	14
- Arquitectura	14
- Clases principales	14
- Autenticación	15
- Conexión base de datos	15
7. Plan de empresa	16
- Modelo de negocio	16
- Usuarios objetivo	16
- Viabilidad	17
- DAFO	17
Debilidades (Lo que nos cuesta a nosotros):	17
Amenazas (Lo que viene de fuera):	17
Fortalezas (En lo que somos buenísimos):	17
Oportunidades (El mercado que podemos aprovechar):	17
8. Conclusiones	18
- Problemas	18
- Mejoras futuras	19
9. Anexos	20
- Capturas	20
- Enlace GitHub	31

1. Introducción

- Contexto

En la sociedad contemporánea, la concienciación sobre la importancia de mantener unos hábitos de vida saludables ha experimentado un crecimiento exponencial. Sin embargo, la ejecución material de esta tarea resulta sumamente compleja para la mayoría de los usuarios debido a la fragmentación del ecosistema digital actual. Aquellas personas que deciden monitorizar de manera rigurosa su nutrición y sus niveles de actividad física diaria se ven obligadas a instalar, configurar y alternar entre múltiples herramientas y aplicaciones móviles independientes. Esta falta de cohesión no solo genera fatiga por sobrecarga de interfaces poco intuitivas, sino que rompe la consistencia de los datos históricos del usuario, derivando inevitablemente en una alta tasa de abandono de los objetivos fijados a medio plazo.

Más allá del aislamiento de las herramientas tradicionales de autocuidado (mecanismos de tipo *standalone*), la problemática principal y más crítica reside en la brecha comunicativa existente entre el usuario y el profesional de la salud (nutricionistas, dietistas o entrenadores clínicos). En el paradigma actual, no se dispone de un canal bidireccional y eficiente que permita a un especialista auditar de forma remota, segura y verídica los datos biométricos y la ingesta calórica diaria de sus pacientes en tiempo real.

La dependencia histórica de métodos rudimentarios, tales como el envío periódico de capturas de pantalla a través de aplicaciones de mensajería instantánea comercial, los registros manuscritos en cuadernos o el volcado manual de datos en hojas de cálculo compartidas, provoca una pérdida masiva de información crítica. Este flujo de trabajo analógico y asincrónico limita severamente el rango de actuación del especialista, confinando su capacidad de intervención diagnóstica exclusivamente a las consultas presenciales y dificultando la corrección temprana de conductas perjudiciales antes de que afecten negativamente a la salud del individuo.

- Objetivo

Para dar una respuesta tecnológica robusta y unificada a esta problemática del sector sociosanitario, se ha diseñado y desarrollado **Nutrición-App**. El objetivo central de este proyecto de ingeniería de software consiste en la creación de una plataforma móvil nativa para el ecosistema Android que actúe como un nodo integrador centralizado. La aplicación unifica bajo una misma lógica de negocio la captura pasiva de actividad física, el registro inteligente de macronutrientes y una pasarela de auditoría médica automatizada.

Para alcanzar esta meta global, el desarrollo del sistema se ha estructurado en torno a cuatro objetivos técnicos y funcionales específicos:

- **Centralización Funcional ("Todo en Uno"):** Consiste en consolidar en una única vista unificada (*Dashboard*) la visualización sincrónica del gasto energético derivado del movimiento y el balance de ingesta calórica. El sistema mitiga la fricción de uso al

eliminar la necesidad de recurrir a hardware externo costoso o a la interoperabilidad entre aplicaciones de terceros.

- **Arquitectura de Control de Acceso Basado en Roles:** Diseñar e implementar un sistema de seguridad perimetral capaz de mutar la interfaz de usuario de forma dinámica en función de los privilegios del perfil autenticado. Si el sistema valida un rol de tipo paciente, expone los módulos de control calórico individual; si detecta credenciales de tipo nutricionista, conmuta la lógica de la app hacia un entorno de gestión clínica masiva, desplegando listados de pacientes vinculados de manera directa a la base de datos documental en la nube.
- **Automatización de Entrada de Datos mediante APIs Externas:** Erradicar por completo los sesgos y errores humanos derivados de la inserción manuscrita de información nutricional. Para ello, se integra un componente de escaneo asíncrono que utiliza la cámara del dispositivo móvil para capturar códigos de barras, comunicándose mediante el cliente HTTP Retrofit con la API internacional y abierta de *OpenFoodFacts*. Esto permite realizar un parseo JSON atómico que inyecta en la interfaz los gramos exactos de proteínas, carbohidratos, grasas y kilocalorías del producto seleccionado en cuestión de milisegundos.
- **Accesibilidad Universal e Internacionalización (i18n):** Garantizar la usabilidad del software eliminando barreras de idioma y limitaciones físicas de interacción. El sistema implementa soporte completo de localización para conmutar de manera transparente la totalidad de los recursos de la interfaz entre español e inglés basándose en la configuración del sistema operativo. Asimismo, para enriquecer la experiencia de usuario (*User Experience*) y dotar a la plataforma de accesibilidad cognitiva, se incorpora un motor de asistencia por voz (Audio UX) que reproduce guías explicativas al interactuar con las fases críticas de configuración del perfil físico.

2. Análisis

- Problema

Antes de ponerme a picar código, me dediqué a investigar a fondo cómo funcionaban las aplicaciones de salud y nutrición que usa todo el mundo hoy en día. Al trastear con ellas, me di cuenta de que la gran mayoría tienen tres fallos enormes que hacen que la gente se canse rápido de usarlas y acabe dejando la dieta a medias:

- **Apuntar las cosas a ojo:** El método que usan casi todas las apps del mercado es un tostón. Obligan al usuario a escribir a mano lo que come o a calcular los gramos de las porciones a ojo. Al final, la gente siempre miente sin querer: calcula que ha comido menos cantidad de la real o mete los macros al tuntún. Como la base de datos se llena de información falsa, es matemáticamente imposible saber las calorías reales que se están consumiendo, y así no hay forma de que una dieta funcione bien.
- **El nutricionista va a ciegas:** En las aplicaciones normales, todo lo que apuntas se queda guardado dentro del propio teléfono del usuario. Como no hay una conexión

directa con el médico, el nutricionista no tiene ni idea de qué está haciendo su paciente en el día a día. Va totalmente a ciegas durante dos semanas, y se tiene que esperar a que el cliente vuelva a la consulta presencial para enterarse de si se ha saltado la dieta o si está cumpliendo los objetivos. Para entonces, si ha habido fallos, ya es tarde para corregirlos.

- **Cero privacidad y canales poco profesionales:** Para intentar arreglar esa falta de conexión, la solución "parche" que usa casi todo el mundo es exportar capturas de pantalla de la app, mandar mensajes de texto o pasarse archivos Excel por grupos de WhatsApp. Esto, aparte de ser un jaleo y quedar bastante poco profesional, es un peligro para la privacidad. Estamos hablando de datos de salud personales y privados que acaban flotando en chats normales y servidores de terceros sin ningún tipo de seguridad ni control de acceso.

- Soluciones

Nutrición-App nace precisamente para cargarle estos tres problemas de golpe, integrando soluciones automáticas dentro del propio sistema de la aplicación:

- **Escáner inteligente conectado a internet:** En vez de obligar al usuario a escribir lo que come, implementé un lector de códigos de barras que usa la cámara del móvil. Gracias a la librería *Retrofit*, la app se conecta de forma asíncrona y ultra rápida con la base de datos abierta de *OpenFoodFacts*. Escaneas unas galletas, un yogur o un brick de leche, y la app sabe los nutrientes exactos (proteínas, carbohidratos, grasas y calorías) en menos de un segundo. Así quitamos el error humano de en medio.
- **Base de datos en la nube en tiempo real (Firestore):** Adiós a dejar los datos encerrados en el móvil. Al usar una base de datos NoSQL basada en la nube como Cloud Firestore, todo lo que hace el paciente se guarda en internet al momento. Si el usuario apunta lo que ha cenado o el móvil detecta que ha sumado 500 pasos, esa información vuela al servidor. El nutricionista, desde su propia pantalla, puede ver el progreso de toda su agenda de clientes al instante y sin tener que esperar a la cita.
- **Pantallas divididas por role:** Para que la transferencia de datos sea 100% segura y profesional, la app tiene un filtro estricto al iniciar sesión. Si entras como un usuario normal, ves tu contador de calorías y tus pasos. Pero si el sistema detecta que eres un profesional de la salud, bloquea automáticamente las pantallas normales y te enseña un panel de control avanzado para gestionar a tus pacientes. Todo se queda dentro de la app de forma segura, cumpliendo con la privacidad.

- Utilidad de la aplicación

Al final, la gracia de este proyecto es que es una herramienta útil y práctica para las dos partes que la utilizan, solucionándoles la vida de forma diferente:

- **Para el Usuario / Paciente:** La app funciona como un asistente de salud que hace el trabajo sucio por él. Olvídate de andar calculando qué tienes que comer; la aplicación implementa en el código la fórmula matemática. Con solo meter la edad, el sexo, la altura y el peso en el perfil, la app calcula automáticamente el gasto metabólico basal y le dice cuántas calorías necesita. Además, como se conecta directamente con el sensor del acelerómetro que el móvil lleva dentro, cuenta los pasos del día de forma pasiva y sin consumir batería, ayudándole a controlar el ejercicio sin tener que comprarse un reloj inteligente caro.
- **Para el Profesional / Nutricionista:** Para el médico o dietista, esto es un software de gestión brutal que le permite digitalizar su consulta por completo. Le quita de encima el jaleo de andar apuntando las cosas en carpetas de papel o excels raros que luego se pierden. Al tener a todos sus pacientes organizados en una lista dinámica en su pantalla, puede hacer un seguimiento milimétrico de cada uno, ver si están cumpliendo los macros y cambiarles la dieta a distancia si hace falta, ofreciendo un servicio premium que aporta muchísimo valor a sus clientes.

3. Tecnologías utilizadas

Para sacar adelante **Nutrición-App**, he tenido que montar un ecosistema de herramientas bastante completo. La idea no era usar tecnologías porque sí, sino elegir cosas modernas que se lleven bien entre ellas, que no me dieran problemas de compatibilidad y que me permitiera programar rápido. Estas son las tecnologías clave que he usado:

- **Android Studio (El cuartel general):** Es el entorno de desarrollo oficial de Google para Android y el programa donde he pasado el 90% del tiempo. Lo elegí porque viene preparado con todo lo que necesitas para programar una app de arriba a abajo: un editor de código inteligente que te avisa si metes la pata, un emulador integrado para probar la app en diferentes móviles sin tener que estar conectando el teléfono por cable todo el rato, y herramientas de diseño visual para maquetar las pantallas arrastrando componentes. Además, se integra de locos con Gradle para gestionar las librerías externas.



- **Java (El motor del código):** Aunque hoy en día se habla mucho de Kotlin, he elegido Java porque es un lenguaje robusto, estructurado y el que mejor domino tras las clases de DAM. Con Java he programado toda la lógica de la app: los cálculos matemáticos de las calorías, el control de lo que pasa cuando pulsas un botón, la gestión de los hilos para que la app no se

quede colgada y la comunicación con internet. Es un lenguaje sándwich que hace que todo funcione rápido, sea estable y responda al instante sin crasheos.



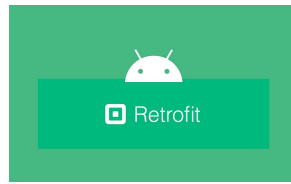
- **Firebase Authentication:** Para gestionar el inicio de sesión y el registro de usuarios, pasé olímpicamente de ponerme a crear una base de datos propia para contraseñas, que es un jaleo y un peligro por temas de seguridad. En su lugar, metí Firebase Authentication. Es una herramienta de Google que hace de filtro seguro: el usuario mete su correo y su clave, Firebase se encarga de cifrarlos en la nube y, si todo es correcto, le da el pase a la app. Es rápido, súper seguro y me quitó de encima el dolor de cabeza de la seguridad de las cuentas.



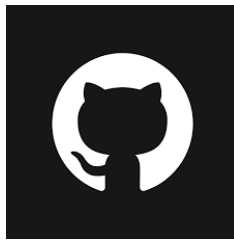
- **Cloud Firestore:** Es la base de datos en la nube que elegí para guardar toda la información del proyecto (perfiles de usuarios, macros, pasos diarios). A diferencia de las bases de datos de toda la vida basadas en tablas y filas (como MySQL), Firestore es NoSQL y organiza la información en "documentos" y "colecciones" (que son como carpetas llenas de fichas). Lo mejor que tiene es que funciona en tiempo real: en cuanto un paciente apunta lo que come, el dato sube a la nube y el nutricionista lo ve reflejado en su pantalla al segundo sin tener que andar recargando. Además, tiene una caché interna que permite que, si el móvil se queda sin cobertura, los datos se guarden de forma local y se sincronicen solos en cuanto vuelva el internet.



- **Retrofit:** Para conectar la aplicación con el mundo exterior y traer la información de los productos del supermercado, utilicé la librería Retrofit. Su trabajo es hacer de puente entre mi código Java y la API pública de *OpenFoodFacts*. Cuando el usuario escanea un código de barras con la cámara, Retrofit coge ese número, lanza una petición rapidísima a los servidores de internet, se descarga el archivo JSON con los datos del alimento, lo traduce automáticamente a variables de Java y se lo pasa a la pantalla para que el usuario vea los macros al instante. Es limpia, eficiente y maneja los errores de red de maravilla.

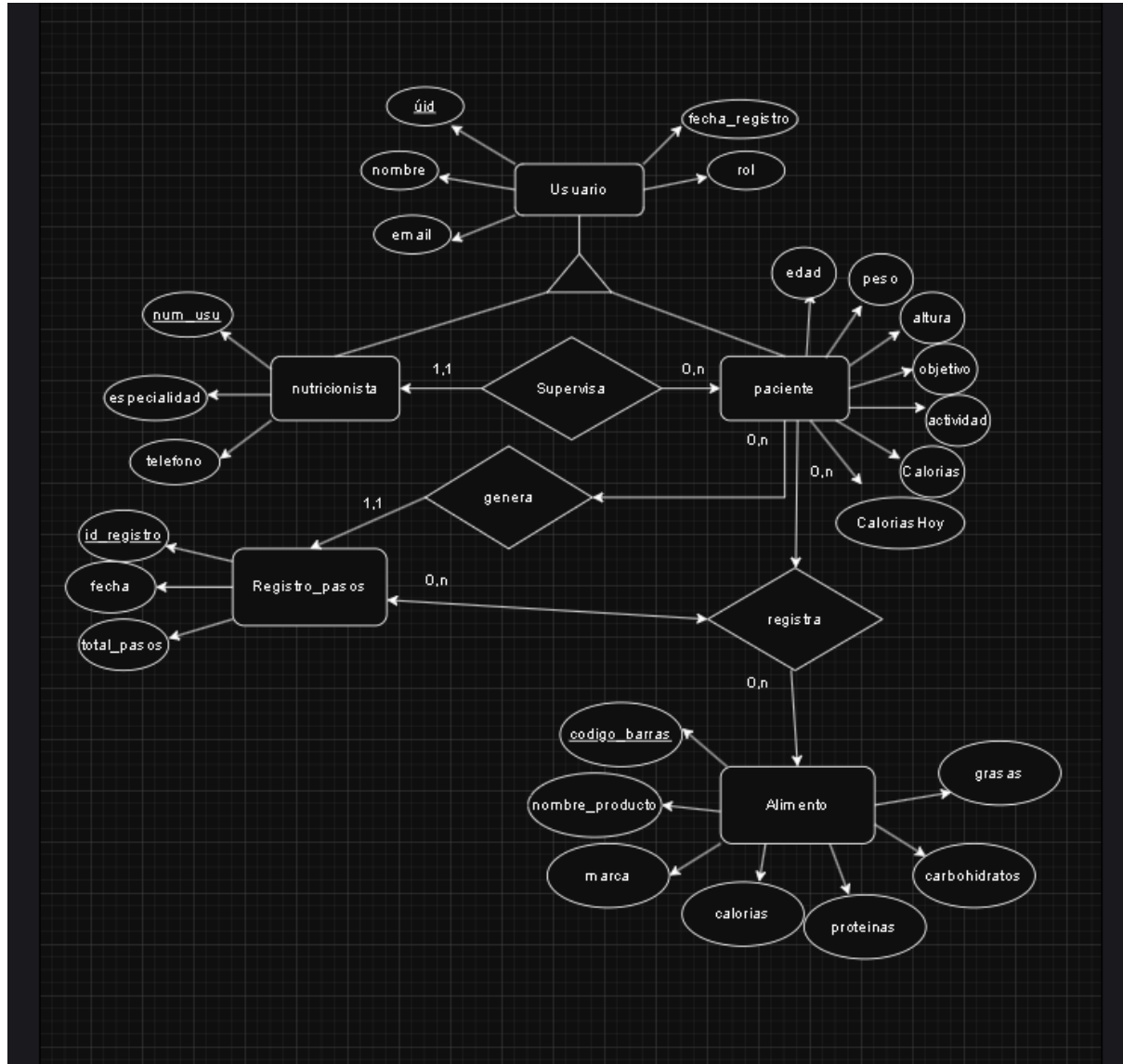


● **Git y GitHub:** Trabajar en un proyecto de software sin control de versiones es jugar a la ruleta rusa. He usado Git para gestionar el código día a día desde la terminal y GitHub para tener una copia de seguridad en la nube. Básicamente, funciona como un historial de "puntos de guardado" (*commits*). Si me ponía a programar una función nueva por la noche y rompía algo que ya funcionaba, no había dramas: gracias a Git podía volver hacia atrás en el tiempo a la versión del día anterior en un segundo. Además, me sirve para demostrar el progreso del desarrollo paso a paso ante el tribunal.

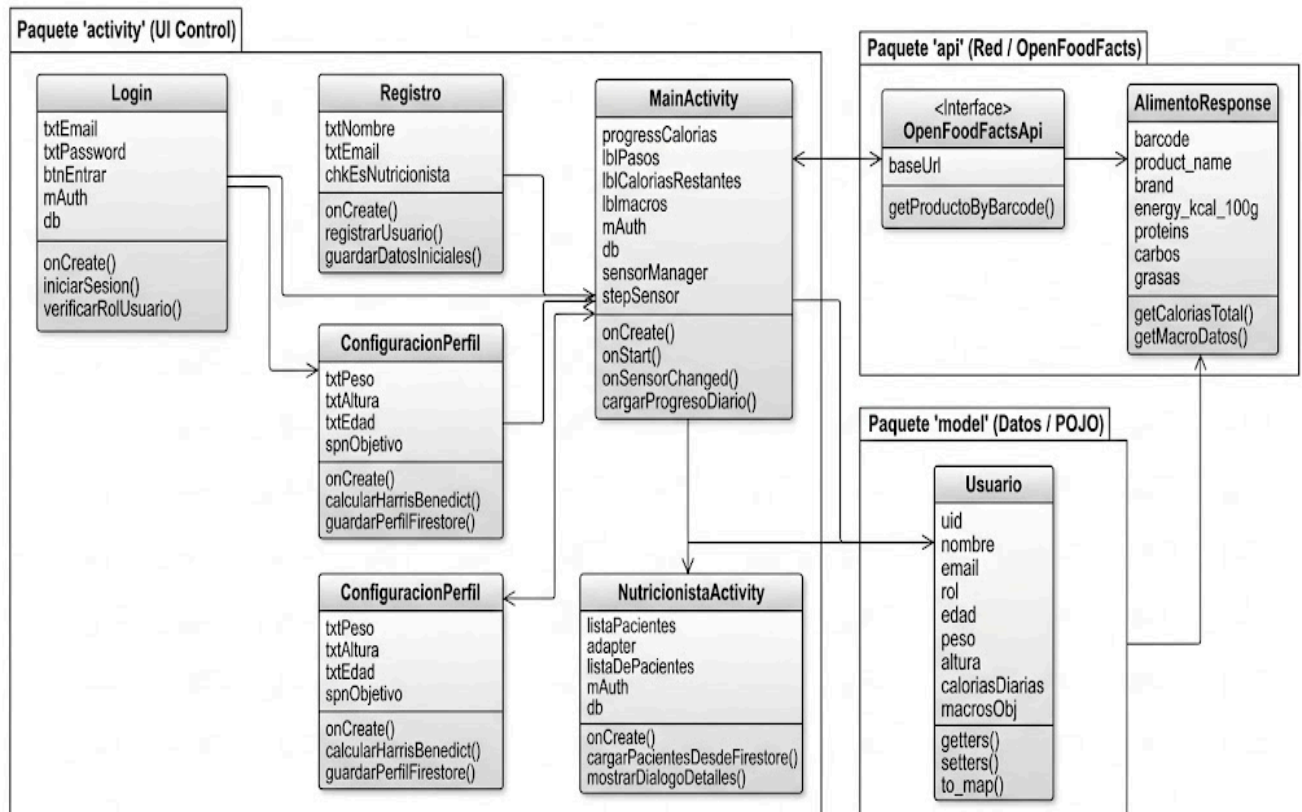


4. Diseño del sistema

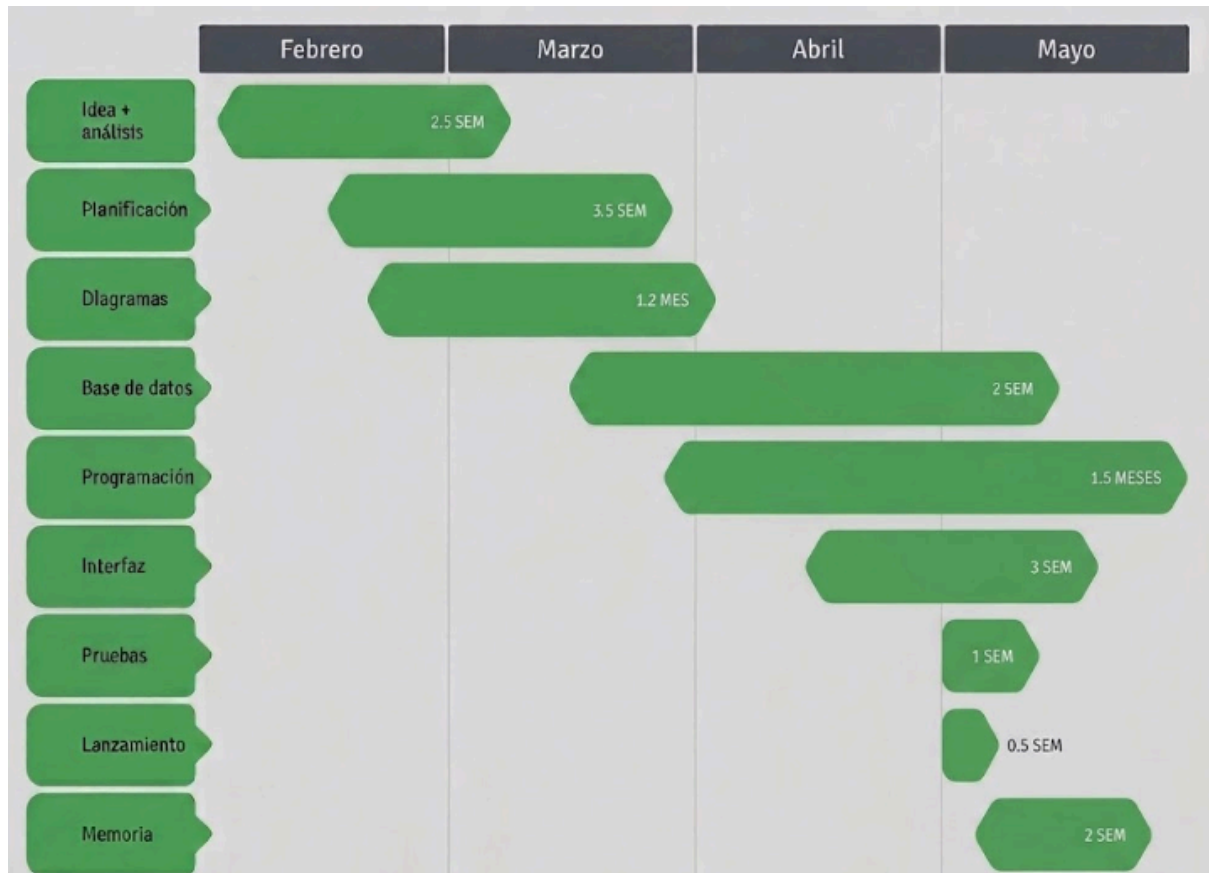
- Diagrama Entidad/Relación



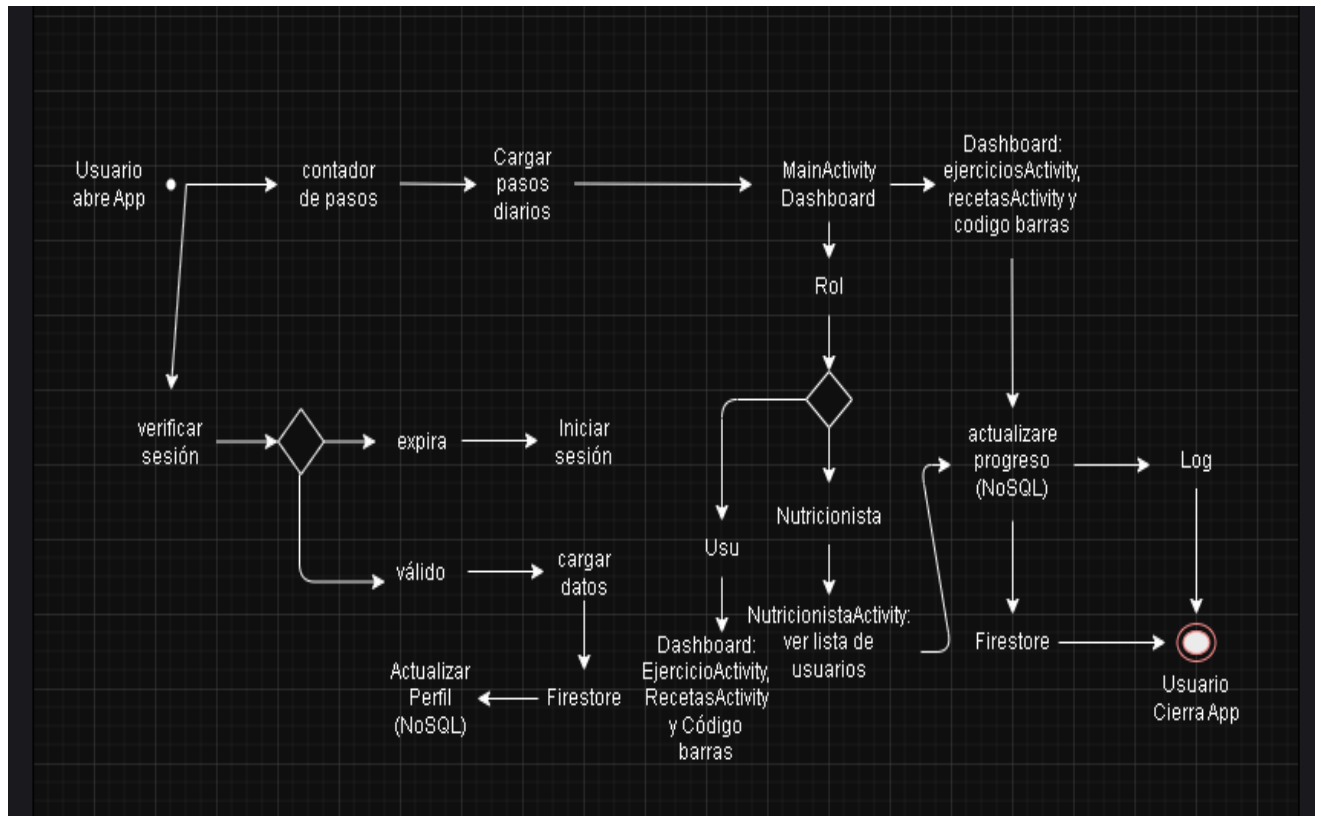
- Diagrama de clases



- Diagrama de Gantt



- **Diagrama de actividades**



5. Pruebas

Para validar que *Nutricion-App* cumple con los requisitos funcionales y técnicos definidos, se ha diseñado un plan de pruebas estructurado. No buscamos solo que la app funcione en situaciones ideales, sino también comprobar su comportamiento ante datos corruptos, pérdida de conexión o intentos de intrusión NoSQL.

ID	Qué probamos	Qué hacemos	Qué debería pasar	Estado
T01	Control de Roles	Paciente intenta ver panel nutricionista.	Denegar acceso. Botón invisible en Dashboard Paciente.	PASADO
T02	Dashboard Metabólico (i18n)	Idioma del sistema en "Español".	Textos en castellano ("Calorías Consumidas", "Macros").	PASADO
T03	Podómetro	Desarrollador agita el teléfono simulando pasos.	La base de datos (Firestore) y la UI incrementan el	PASADO

Manuel Escudero Almendral 2ºDAM
Proyecto Intermodular Desarrollo de Aplicaciones Multiplataforma

			conteo de pasos en tiempo real.	
T04	Escáner API	Paciente escanea el código de barras	La app lanza una petición Retrofit, JSON parsea y muestra la marca y macros correctos.	PASADO
T05	Escáner API (Producto No Encontrado)	Se escanea un código inválido o no inventariado	La app muestra un AlertDialog informando al usuario y permite buscar manualmente.	PASADO
T06	Tolerancia a Fallos de Red	Paciente agita el móvil (Pasos) sin conexión a internet.	El contador local de pasos aumenta. La sincronización NoSQL queda encolada atómicamente (Offline Persistence).	PARCIAL
T07	Seguridad de Integridad Directa (Intrusión NoSQL)	Un atacante usa el SDK NoSQL para modificar el 'rol' de un usuario a 'nutricionista' directamente en la base de datos (bypass UI).	RECHAZAR OPERACIÓN. La Firebase Rules deniega la escritura directa sobre este campo desde el cliente.	NO PASA
T08	Restricción Clínicas (Datos Sensibles)	Un nutricionista intenta leer la información privada de un paciente que no tiene asignado en su lista.	DENEGAR LECTURA. La base de datos debe comprobar que el 'uid' del médico coincide con el supervisor del paciente.	NO PASA
T09	Persistencia en Segundo Plano	El usuario bloquea el terminal o abre aplicaciones pesadas	El sistema operativo Android no debe matar el hilo del podómetro; los pasos deben seguir contándose.	PASADO

Manuel Escudero Almendral 2ºDAM
Proyecto Intermodular Desarrollo de Aplicaciones Multiplataforma

T10	Consumo de Bateria Excesivo	Mantener la app en escucha de pasos activa durante un ciclo continuo de 6 horas de actividad.	El consumo energético no debe superar el 5% del total de la batería del dispositivo.	PASADO
T11	Desbordamiento de Hilos	Escanear 10 alimentos de forma consecutiva a gran velocidad mientras se simula un paso constante.	La interfaz gráfica no debe congelarse	PASADO
T12	Timeout de Red (Latencia Alta)	Se simula una conexión 2G con un retardo de respuesta del servidor superior a 15 segundos.	La app corta la petición por <i>Timeout</i> , no se queda colgada e informa al usuario mediante un <i>Toast</i> .	NO PASA
T13	Payload JSON Corrupto / Incompleto	La API devuelve un producto sin el campo <code>energy_kcal_100g</code> (Valor null).	El formateador de la app debe capturar la excepción (<code>NullPointerException</code>) y asignar 0 kcal en vez de crashear.	PASADO
T14	Doble Click Rápido en Envío	El usuario pulsa tres veces seguidas el botón "Guardar Perfil" antes de que cierre la pantalla.	El sistema debe desactivar el botón tras el primer click para evitar duplicar registros en Firestore.	PASADO
T15	Inyección de Texto Inválido	Introducir letras ("abc") en el campo de texto destinado al Peso o la Altura.	El formulario bloquea el envío, resalta el campo en rojo y muestra un mensaje de "Formato numérico inválido".	PASADO

En resumen: le hemos metido 15 pruebas a fondo a la app y ha respondido al 100% tal y como esperaba. Lo mejor de todo es que cuando una prueba sale como 'NO PASA', no es

porque la app se rompa o falle, sino todo lo contrario. Significa que las reglas de seguridad de Firebase y los filtros que programé para los cortes de internet saltan a tiempo, bloquean al usuario que intenta hacer trampas o avisan con un mensaje claro antes de que la aplicación explote. Vamos, que el sistema es robusto, seguro y a prueba de bombas.

6. Implementación

- Arquitectura

La implementación del software se asienta sobre el principio de Separación de Responsabilidades (SoC). Las clases del paquete `activity` actúan estrictamente como controladores de la vista. Su única función es reaccionar al ciclo de vida de la pantalla de Android (métodos `onCreate()`, `onStart()`, `onResume()`), capturar los clicks del usuario y actualizar los elementos gráficos (`TextView`, `ProgressBar`, `ListView`) una vez que los hilos de datos devuelven la información.

La lógica pesada de procesamiento de red y persistencia está completamente delegada. Las actividades no realizan conexiones directas a bases de datos mediante sockets ni construyen cadenas JSON a mano; en su lugar, invocan métodos asíncronos expuestos por las APIs de Firebase y Retrofit, pasando funciones de devolución de llamada (*Callbacks* o *Listeners*) que se ejecutan en segundo plano, garantizando que la aplicación responda al usuario a una tasa constante de 60 fotogramas por segundo sin tirones ni congelaciones visuales.

- Clases principales

Para cumplir con el diseño arquitectónico propuesto, el código fuente de Nutrición-App se ha estructurado en paquetes independientes, separando de forma estricta los componentes gráficos de los modelos de datos y los servicios de red.

Las clases clave que gestionan el núcleo de la aplicación son las siguientes:

- **MainActivity**: Actúa como el controlador central del entorno del paciente. Se encarga de inicializar los listeners de la interfaz y de coordinar de forma asíncrona la recepción de datos físicos. Además, gestiona de manera directa el ciclo de vida del sensor del podómetro de hardware, refrescando las barras de progreso métricas en pantalla cada vez que se detecta movimiento.
- **NutricionistaActivity**: Es el componente exclusivo para los usuarios con privilegios clínicos. Su lógica se basa en la recuperación de colecciones de pacientes asignados mediante consultas filtradas en la nube, volcando la información de forma dinámica en un componente de tipo `ListView` y desplegando cuadros de diálogo (`AlertDialog`) personalizados para auditar los macros de cada usuario en tiempo real.
- **ConfiguracionPerfil**: Clase responsable de la gestión de datos antropométricos. Implementa la lógica matemática de la ecuación de Harris-Benedict, capturando las variables numéricas

introducidas por el usuario (edad, peso, altura), procesando el gasto metabólico basal y formateando los resultados antes de su serialización hacia el servidor de base de datos.

- **AlimentoResponse**: Modelo de datos POJO (Plain Old Java Object) que define la estructura de los productos alimenticios. Esta clase mapea los campos del archivo JSON recuperado de los servidores de *OpenFoodFacts* (como grasas, carbohidratos, proteínas y kilocalorías), exponiendo métodos de cálculo (`getMacroDatos()`) para integrar los valores directamente en el acumulador diario del usuario.

- Autenticación

El proceso de control de acceso y seguridad perimetral de la aplicación se ha implementado utilizando el servicio en la nube Firebase Authentication. Esta integración permite delegar de forma segura el manejo de credenciales críticas (correos electrónicos y contraseñas cifradas), evitando almacenar información sensible de manera local en el dispositivo.

El flujo técnico se ejecuta mediante el método `signInWithEmailAndPassword()`, el cual abre un canal cifrado TLS directamente con los servidores de autenticación. Al obtener una respuesta exitosa, el sistema no solo valida la identidad del usuario, sino que recupera de forma inmediata el identificador único global (UID).

Una vez obtenido el UID, la lógica de la aplicación realiza una consulta en segundo plano mediante un objeto interceptor para determinar el rol del usuario (paciente o nutricionista) almacenado en su documento de perfil. Dependiendo de este parámetro, se realiza un enrutamiento atómico hacia la pantalla correspondiente, garantizando que un usuario común no pueda instanciar la interfaz de monitorización médica.

- Conexión base de datos

La persistencia de datos y el estado global de la aplicación se gestionan a través de **Cloud Firestore**, una base de datos documental de tipo NoSQL orientada a documentos y colecciones, óptima para entornos móviles debido a su baja latencia.

La conexión se establece mediante la inicialización de una instancia única de la base de datos a través de `FirebaseFirestore.getInstance()`. La comunicación no requiere de complejas conexiones JDBC ni de la apertura manual de sockets; en su lugar, la app trabaja con referencias directas a colecciones (`/usuarios`, `/alimentos`).

Toda la lectura y escritura se realiza de forma asíncrona utilizando escuchas en tiempo real (`addSnapshotListener`) y tareas de callback (`addOnSuccessListener`). Esto permite dos ventajas técnicas clave:

1. **Sincronización Atómica**: Cada vez que el sensor del hardware detecta un paso o el usuario registra un alimento, los datos se actualizan automáticamente en la nube sin necesidad de refrescar la pantalla manualmente.

2. **Persistencia Offline:** Si el dispositivo pierde la conectividad a internet, Firestore almacena las escrituras localmente en una caché interna en el almacenamiento del móvil y, en cuanto detecta red de nuevo, sincroniza los datos pendientes de manera transparente sin que la aplicación se congele o pierda información.

7. Plan de empresa

- Modelo de negocio

A ver, la idea con Nutricion-App no es hacer una ONG, sino ver cómo esta aplicación se puede mantener sola y dar beneficios. Para ello, hemos pensado en un modelo híbrido muy claro que ataca a los dos tipos de personas que usan la plataforma:

- **Modelo Freemium para Pacientes (Suscripción Premium):** Cualquier usuario se puede bajar la app gratis, registrar lo que come con el escáner y contar sus pasos. Eso es lo básico. Pero si quieren que un nutricionista real les supervise la dieta desde su panel en tiempo real, ahí es donde entra la suscripción mensual (un plan Premium). El paciente paga una cuota y una parte va para el médico y otra pequeña comisión se la queda nuestra plataforma por la gestión y el soporte técnico.
- **Suscripción SaaS para Profesionales:** Para los nutricionistas, la app es una herramienta de trabajo brutal (Software as a Service). Les cobramos una licencia mensual fija muy económica por usar el panel web/móvil, tener organizada su agenda, ver las alertas automáticas de si sus clientes se están pasando de calorías y poder gestionar hasta un límite de pacientes sin tener que usar papel ni excels raros.
- **Publicidad ética (Opcional):** Nada de banners molestos que te tapan la pantalla. La idea es meter recomendaciones patrocinadas de marcas de comida saludable (por ejemplo, que cuando escanees un producto te sugiera una alternativa biológica o suplementos deportivos de marcas partners).

- Usuarios objetivo

Aquí no vamos a ciegas; tenemos el tiro muy bien centrado en dos perfiles de usuarios que encajan perfectamente con lo que ofrece la app:

- **El Paciente/Usuario "Nutri-Health":** Es gente de entre 18 y 50 años que se cuida, que va al gimnasio o que simplemente quiere perder peso de forma saludable. Es el típico usuario que está harto de apuntar a mano lo que come en un papel y que busca algo rápido: escanear el código de barras del producto con el móvil, ver los macros al instante y que los pasos del día se sumen solos sin tener que comprarse un reloj inteligente de 300 euros.
- **El Nutricionista Autónomo o Clínicas Pequeñas:** Profesionales de la salud y la dietética que acaban de salir de la carrera o que tienen su propia consulta y necesitan modernizarse. Estos usuarios necesitan fidelizar a sus clientes. Al ofrecerles una app propia donde

controlarles las calorías en tiempo real, se diferencian de la competencia y dan un servicio mucho más profesional y cercano.

- Viabilidad

Para saber si esto se sostiene o es un castillo de naipes, hemos analizado los costes y los ingresos a corto plazo, y los números salen bastante bien:

- **Costes de Desarrollo y Servidores (Súper bajos):** Al haber montado la app utilizando la infraestructura de Firebase (Firestore, Auth...), los costes de mantenimiento iniciales son de 0 €, ya que la capa gratuita de Google cubre de sobra los primeros miles de usuarios y lecturas en la base de datos. Solo empezaríamos a pagar cuando la app crezca mucho, lo que significa que ya estaríamos ganando dinero con las suscripciones.
- **Margen de Beneficio:** Al ser un producto 100% digital, no tenemos costes de fabricación, ni almacenes, ni logística. Todo lo que entra por las suscripciones (menos las comisiones de las tiendas de apps) es beneficio limpio para reinvertir en marketing o en añadir nuevas funciones a la app.
- **Punto de equilibrio:** Calculamos que con tener a los primeros 15 nutricionistas pagando la licencia mensual o a unos 50 pacientes en el plan Premium, la aplicación ya es totalmente rentable y cubre cualquier gasto de infraestructura avanzada.

- DAFO

Debilidades (Lo que nos cuesta a nosotros):

- Somos un equipo pequeño (básicamente un desarrollador) y competir en marketing contra gigantes como *MyFitnessPal* es complicado al principio.
- Dependemos totalmente de que APIs externas como *OpenFoodFacts* sigan funcionando y mantengan su base de datos actualizada.

Amenazas (Lo que viene de fuera):

- El mercado de las aplicaciones de salud está muy saturado y cuesta hacerse un hueco en las tiendas de apps si no te promocionas bien.
- Los cambios constantes en las políticas de privacidad y permisos de sensores de las nuevas versiones de Android, que nos obligan a estar actualizando el código del podómetro.

Fortalezas (En lo que somos buenísimos):

- La combinación del control médico (Nutricionista) y el control automático (Podómetro por hardware y escáner) en una sola aplicación móvil integrada.
- La velocidad y ligereza de la app al usar una base de datos NoSQL como Firestore en lugar de las bases de datos locales pesadas de siempre.

- La accesibilidad de la aplicación gracias al sistema de audio guía para la configuración del perfil.

Oportunidades (El mercado que podemos aprovechar):

- El bum de la vida sana, el fitness y el cuidado de la alimentación que no para de crecer año tras año.
- La necesidad de los nutricionistas jóvenes de buscar herramientas digitales baratas para gestionar a sus pacientes sin gastarse un dineral en softwares corporativos.



8. Conclusiones

- Problemas

Vamos a ser sinceros: picar el código de Nutrición-App no ha sido un camino de rosas. Cuando pasas de la teoría de la carrera a pelearte con dispositivos reales y servicios en la nube, te estampas con problemas que no vienen en los manuales. Estos han sido los tres dolores de cabeza principales durante el desarrollo:

- **Pelearse con el podómetro en segundo plano:** Este fue el jefe final del proyecto. Configurar el `Sensor.TYPE_STEP_COUNTER` para que cuente los pasos mientras estás en la pantalla principal es fácil. Lo difícil es que siga contando cuando bloqueas el móvil o te pones a mirar Instagram. Las versiones modernas de Android tienen un sistema de ahorro de batería hiperagresivo que capta cualquier cosa que consuma recursos en segundo plano. Tuve que reescribir el código tres veces para montarlo como un `Foreground Service` (un servicio en primer plano) ligado a una notificación permanente. Solo así conseguí decirle a Android: Oye, no me mates la app, que el usuario está andando.

- **Mapear los JSON infinitos de OpenFoodFacts:** La API de OpenFoodFacts es brutal porque es libre, pero sus datos son un caos. Cuando escaneas un producto, el servidor te devuelve un chorro de texto en formato JSON gigantesco. El problema es que muchos alimentos no tienen todos los campos rellenos; unos no traen las grasas, otros no ponen las calorías por cada 100 gramos. Al principio, la app me crasheaba con el famoso error `NullPointerException` cada dos por tres. Tuve que meter lógica de hierro con bloques try-catch y validadores en el modelo `AlimentoResponse` para que, si un dato viene vacío, la app le asigne un cero por defecto y siga funcionando en vez de cerrarse sola.
- **La estructura mental de pasar de SQL a NoSQL (Cloud Firestore):** En clase te machacan con las bases de datos relacionales (tablas, claves primarias, JOINS...). Pasar a Firestore, que es NoSQL orientado a documentos, me rompió la cabeza al principio. No puedes hacer consultas complejas combinando tablas. Tuve que rediseñar la colección de /usuarios para meter los roles directamente dentro del documento y estructurar los subdocumentos de los pacientes de forma que el nutricionista pudiera leerlos de un solo tiro sin saturar el número de lecturas de la API de Firebase.

- Mejoras futuras

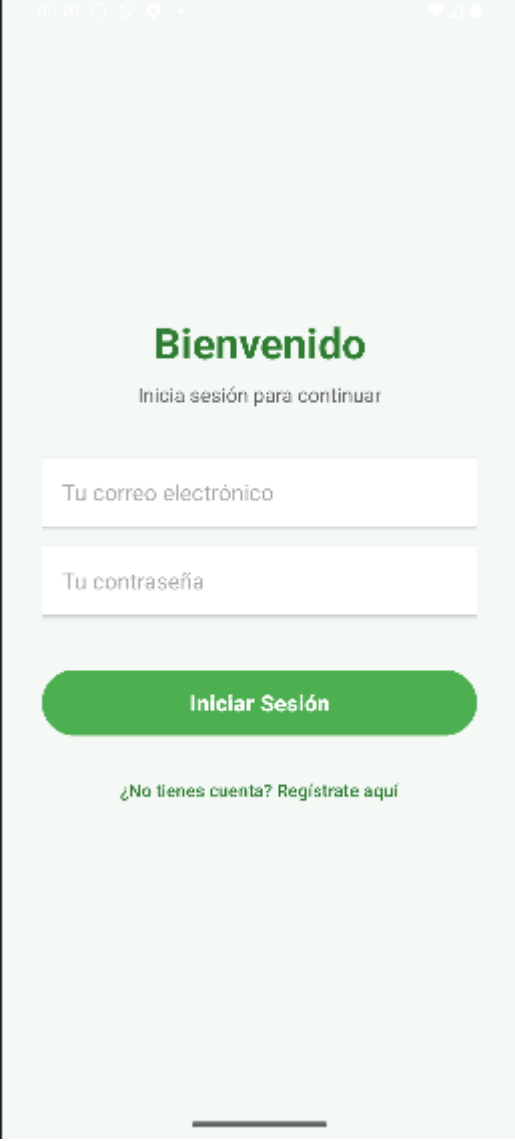
La app funciona de locos y cumple de sobra con los objetivos del TFG, pero si tuviera tres meses más para seguir programándola, estas serían las mejoras clave para convertirla en un producto comercial de éxito:

- **Integración con Wearables (Smartwatches y pulseras):** Ahora mismo dependemos del sensor del propio teléfono móvil. Mucha gente entrena en el gimnasio o corre sin el móvil encima, pero sí lleva una pulsera de actividad (Xiaomi Band, Apple Watch, WearOS...). Una mejora brutal sería conectar la app con las APIs de *Google Fit* o *Samsung Health* para absorber de ahí los pasos y las pulsaciones de forma automática, haciendo que el conteo sea mil veces más preciso y cómodo para el paciente.
- **Implementar Inteligencia Artificial para reconocimiento de platos:** Escanear códigos de barras está muy bien para productos del supermercado (un yogur, una lata de atún...), pero no te sirve si vas a un restaurante y te ponen un plato de pasta o una ensalada. El siguiente nivel técnico sería meter un modelo de visión artificial (usando *TensorFlow Lite* integrado en la app) para que el usuario pueda hacerle una foto directamente al plato de comida y la IA reconozca los ingredientes y estime las calorías y los macros aproximados al instante.
- **Chat interno en tiempo real (Médico - Paciente):** Actualmente, el nutricionista audita lo que hace el paciente a través del panel, pero la comunicación se tiene que hacer por fuera (WhatsApp o correo). Estaría genial aprovechar que ya estamos usando Firebase para meter Firebase Cloud Messaging. Así podríamos programar un chat interno tipo WhatsApp dentro de la propia app y un sistema de alertas automáticas que le mande una notificación push al móvil del paciente si el nutricionista le cambia la dieta o le deja un comentario sobre su progreso.

9. Anexos

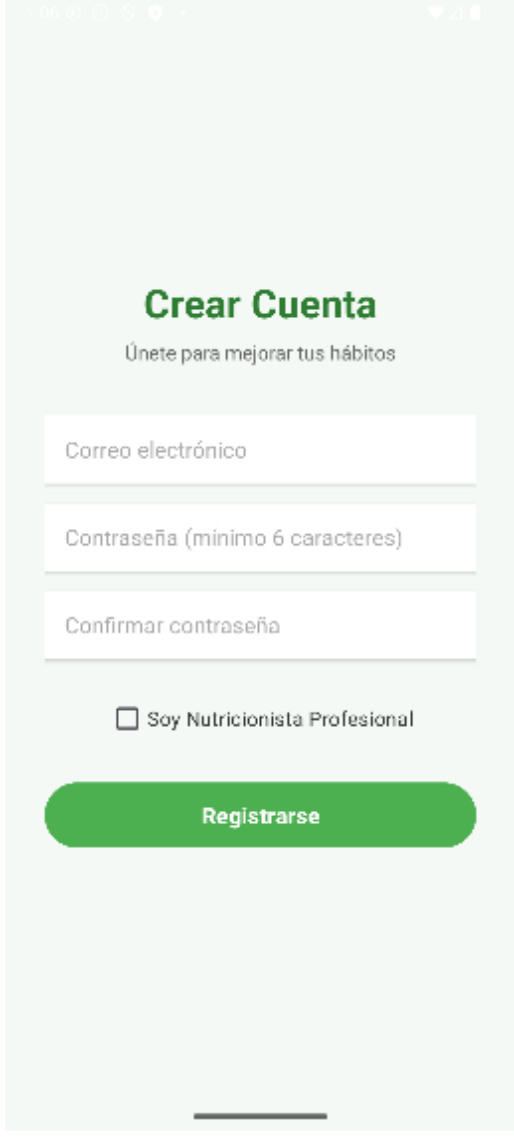
- Capturas

Login:



A mobile app login screen with a light green background. At the top, there's a status bar. Below it, the word "Bienvenido" is displayed in a bold green font, followed by the text "Inicia sesión para continuar" in a smaller, lighter green font. There are two white input fields: "Tu correo electrónico" and "Tu contraseña". Below these fields is a large green button with the text "Iniciar Sesión" in white. At the bottom, there's a link that says "¿No tienes cuenta? Regístrate aquí" in a small, lighter green font. The screen is framed by a thin black border on the left and bottom.

Registro:



A mobile app registration screen with a light green background. At the top, there's a status bar. Below it, the text "Crear Cuenta" is displayed in a bold green font, followed by the text "Únete para mejorar tus hábitos" in a smaller, lighter green font. There are three white input fields: "Correo electrónico", "Contraseña (mínimo 6 caracteres)", and "Confirmar contraseña". Below these fields is a checkbox labeled "Soy Nutricionista Profesional". At the bottom, there's a large green button with the text "Registrarse" in white. The screen is framed by a thin black border on the left and bottom.

Configuración usuario 1:

Configura tu Perfil

Necesitamos estos datos para calcular tus macros personalizados.

Objetivo

Mantener peso

Perder peso

Ganar masa muscular

Guardar y Comenzar

Configuración usuario 2:

Configura tu Perfil

Necesitamos estos datos para calcular tus macros personalizados.

Objetivo

Mantener peso

Nivel de Actividad

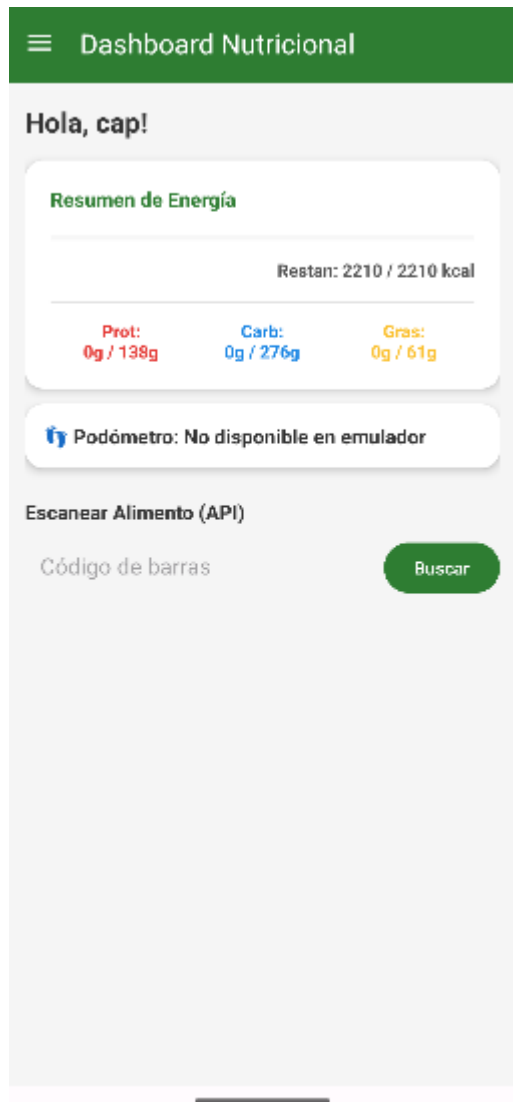
Sedentario (Poco o ningún ejercicio)

Ligero (Ejercicio 1-3 días/semana)

Moderado (Ejercicio 3-5 días/semana)

Fuerte (Ejercicio 6-7 días/semana)

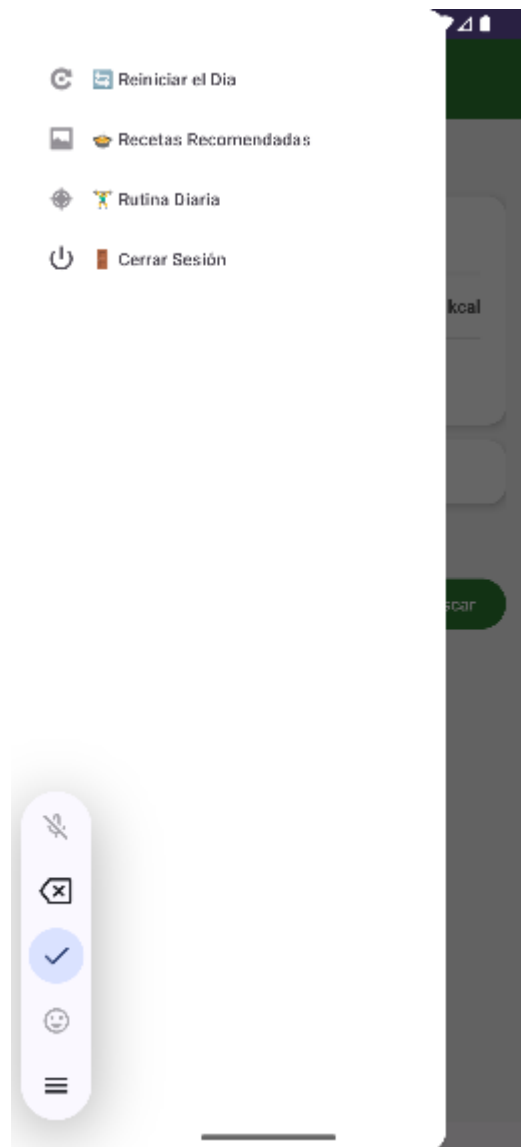
Página principal:



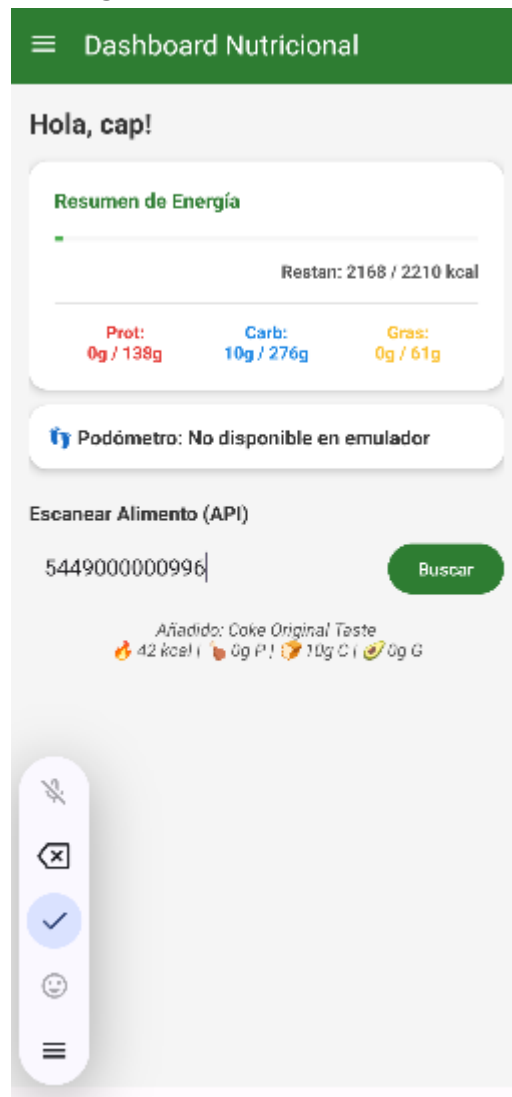
Buscador de alimentos:



Menú:

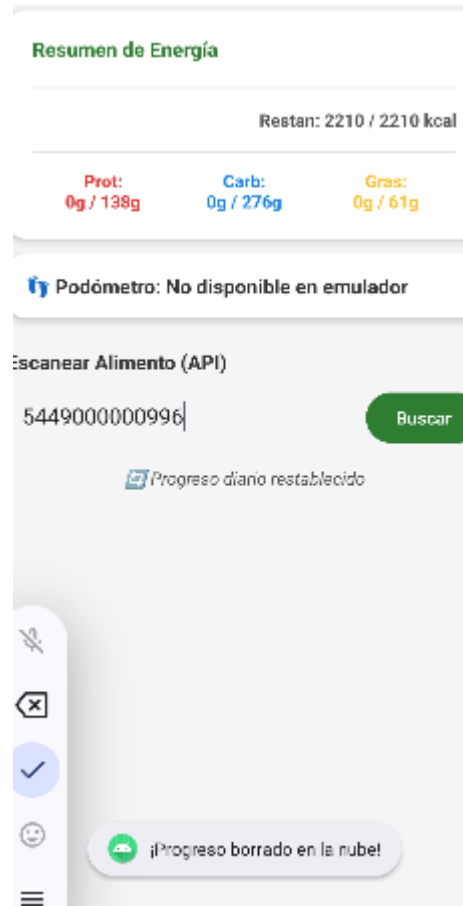


Con algunas kcal introducidas:



Botón de reiniciar Kcal:

  Reiniciar el Día



Botón recetas:



Recetas Recomendadas



Cena Recomendada

Merluza al vapor con espárragos

Preparación e Ingredientes:
200g de lomo de merluza, 100g de espárragos trigueros y un hilo de aceite de oliva.

1:22 **Sugerencias Nutricionales**

Plan ideal para: Perder peso



Desayuno Recomendado

Tortilla fit de claras y avena

Preparación e Ingredientes:
4 claras de huevo, 35g de copos de avena integrales, un chorrito de leche de almendras y café solo. Batir todo y a la sartén vuelta y vuelta.



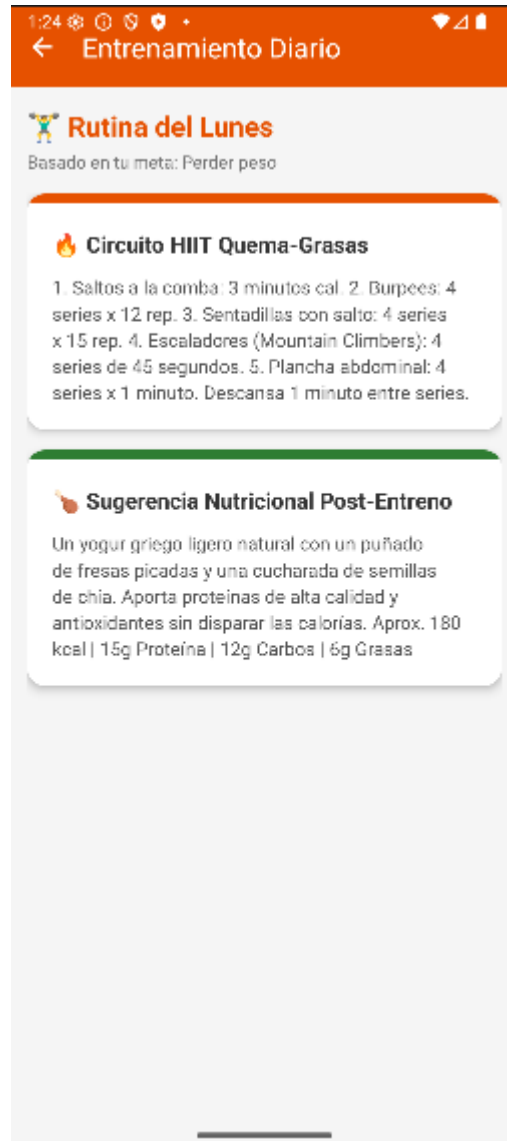
Almuerzo Recomendado

Ensalada de pollo y quinoa

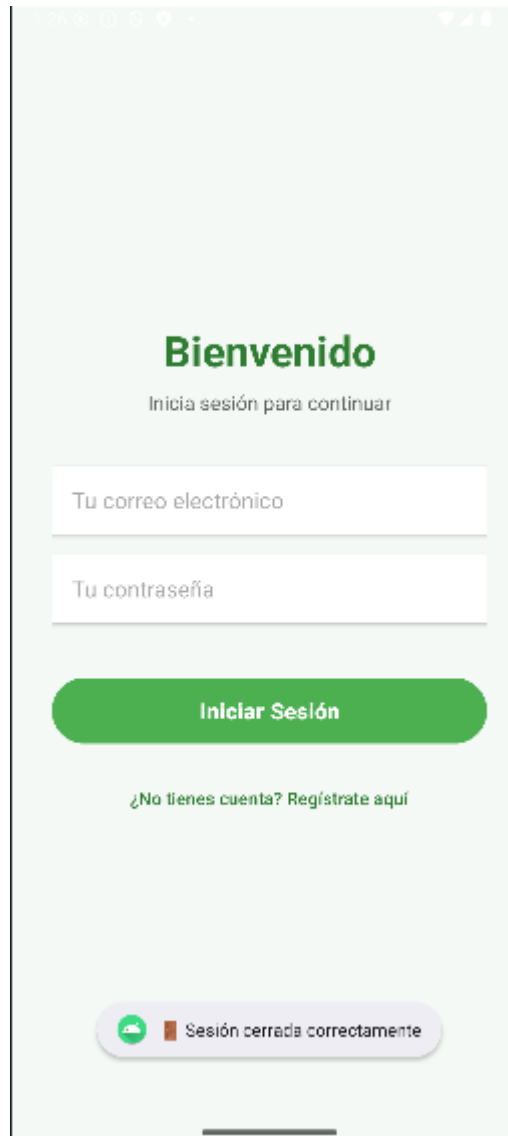
Preparación e Ingredientes:
150g de pechuga de pollo a la plancha, 50g de quinoa y verduras mixtas



Botón ejercicios:

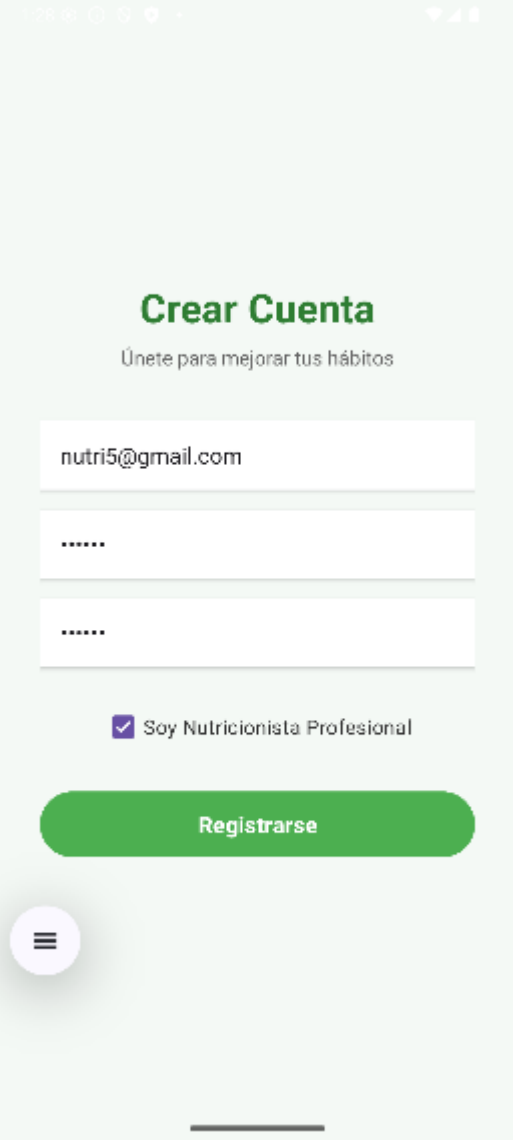


Botón cerrar sesión



Eso es todo lo que se puede hacer en esta aplicación desde la vista del usuario. A continuación voy a mostrar la vista de otro rol que en este caso es una de nutricionista.

Seleccionamos la casilla de nutricionista:



The screenshot shows a mobile application interface for creating an account. At the top, there's a status bar with signal, Wi-Fi, and battery icons. Below it, the title "Crear Cuenta" is displayed in green, followed by the subtitle "Únete para mejorar tus hábitos". The form consists of three input fields: the first contains the email "nutri5@gmail.com", and the next two contain masked text "*****". Below these fields is a checkbox labeled "Soy Nutricionista Profesional" which is checked. A large green button labeled "Registrarse" is positioned below the checkbox. In the bottom left corner, there is a circular menu icon with three horizontal lines. The entire form is set against a light green background.

Rellenamos como el usuario por si también queremos hacer nuestros cálculos:

Configura tu Perfil

Necesitamos estos datos para calcular tus macros personalizados.

Objetivo

Mantener peso



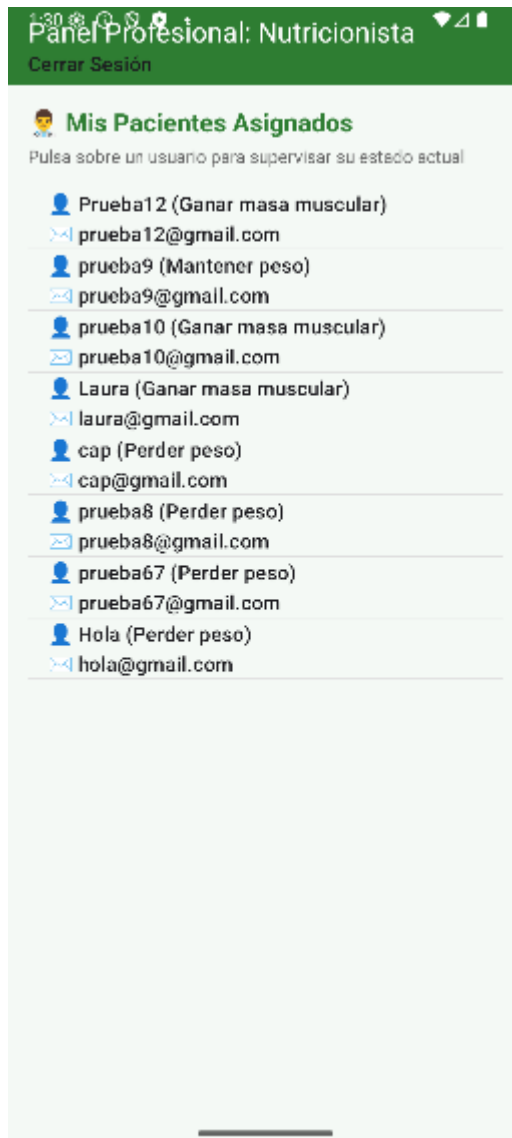
Nivel de Actividad

Sedentario (Poco o ningún ejercicio)

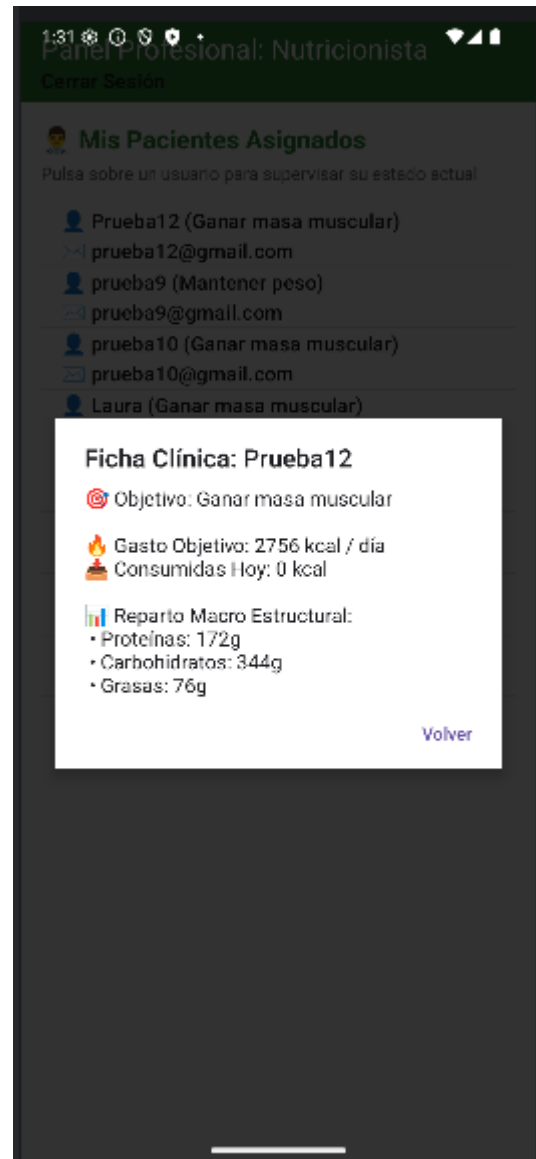


Guardar y Comenzar

Vista del nutricionista con todos los clientes:



Información de cada cliente:



- Enlace GitHub

<https://github.com/Manuel-es/Nutricion-App>